

Files

Objectives

After studying this section, you should be able to:

- ◆ What is a file?
- ◆ How are data stored in files?
- ◆ How to access data in files?

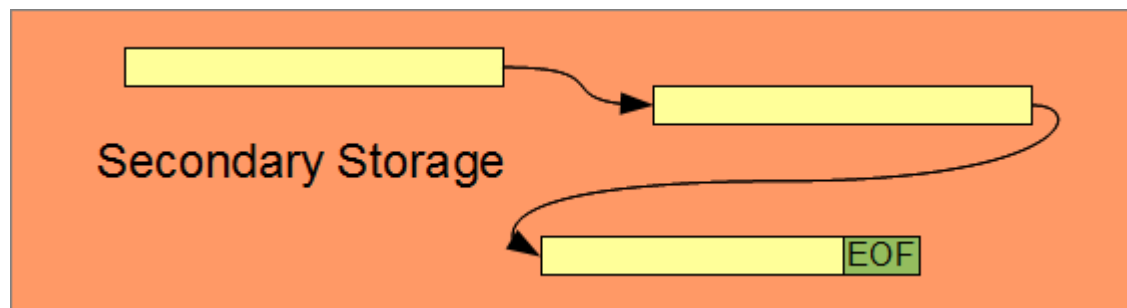
Contents

1. What is a file?
2. Why do we need File Handling in C?
3. Types of Files in C
4. C File Operations: Open, Close, Create, Read, Write, ...

1. What is a file?

What is a file?

- ◆ A file is a named area of secondary storage.
- ◆ The file may be fragmented; that is, it may consist of several parts stored at different non-contiguous locations in secondary memory.
- ◆ A file does not necessarily occupy contiguous space on the storage device.



- ◆ The byte is the fundamental storage unit of a file. The distinguishing feature of a file is the **end-of-file** mark. We refer to this mark as **EOF** (**EOF** typically has the value -1)

2. Why do we need File Handling in C?

Why do we need File Handling in C?

- ◆ The operations using the C program are done on a prompt/terminal which is not stored anywhere.
- ◆ The output is deleted when the program is closed.
- ◆ However, in the software industry, most programs are written to store the information fetched from the program. The use of file handling is exactly what the situation calls for.



Why do we need File Handling in C?

- ◆ **Reusability:** The data stored in the file can be accessed, updated, and deleted anywhere and anytime providing high reusability.
- ◆ **Portability:** Without losing any data, files can be transferred to another in the computer system. The risk of flawed coding is minimized with this feature.
- ◆ **Efficient:** A large amount of input may be required for some programs. File handling allows you to easily access a part of a file using few instructions which saves a lot of time and reduces the chance of errors.
- ◆ **Storage Capacity:** Files allow you to store a large amount of data without having to worry about storing everything simultaneously in a program.

3. Types of Files in C

Type of Files in C

- ◆ A file can be classified into two types based on the way the file stores the data. They are as follows:
 - **Text Files**
 - **Binary Files**



file.txt



file.bin

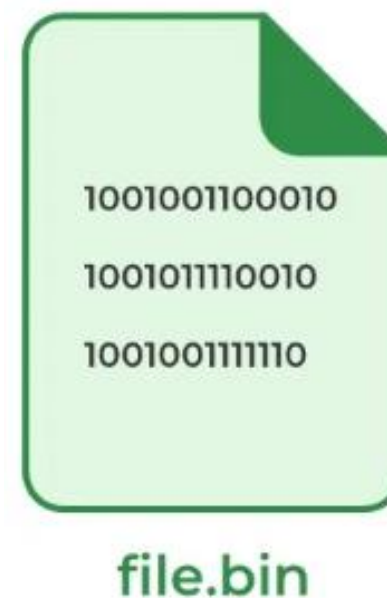
Text Files

- ◆ A text file contains data in the **form of ASCII characters** and is generally used to store a stream of characters.
 - Each line in a text file ends with a new line character (`'\n'`).
 - It can be read or written by any text editor.
 - They are generally stored with **.txt** file extension.
 - Text files can also be used to store the source code.

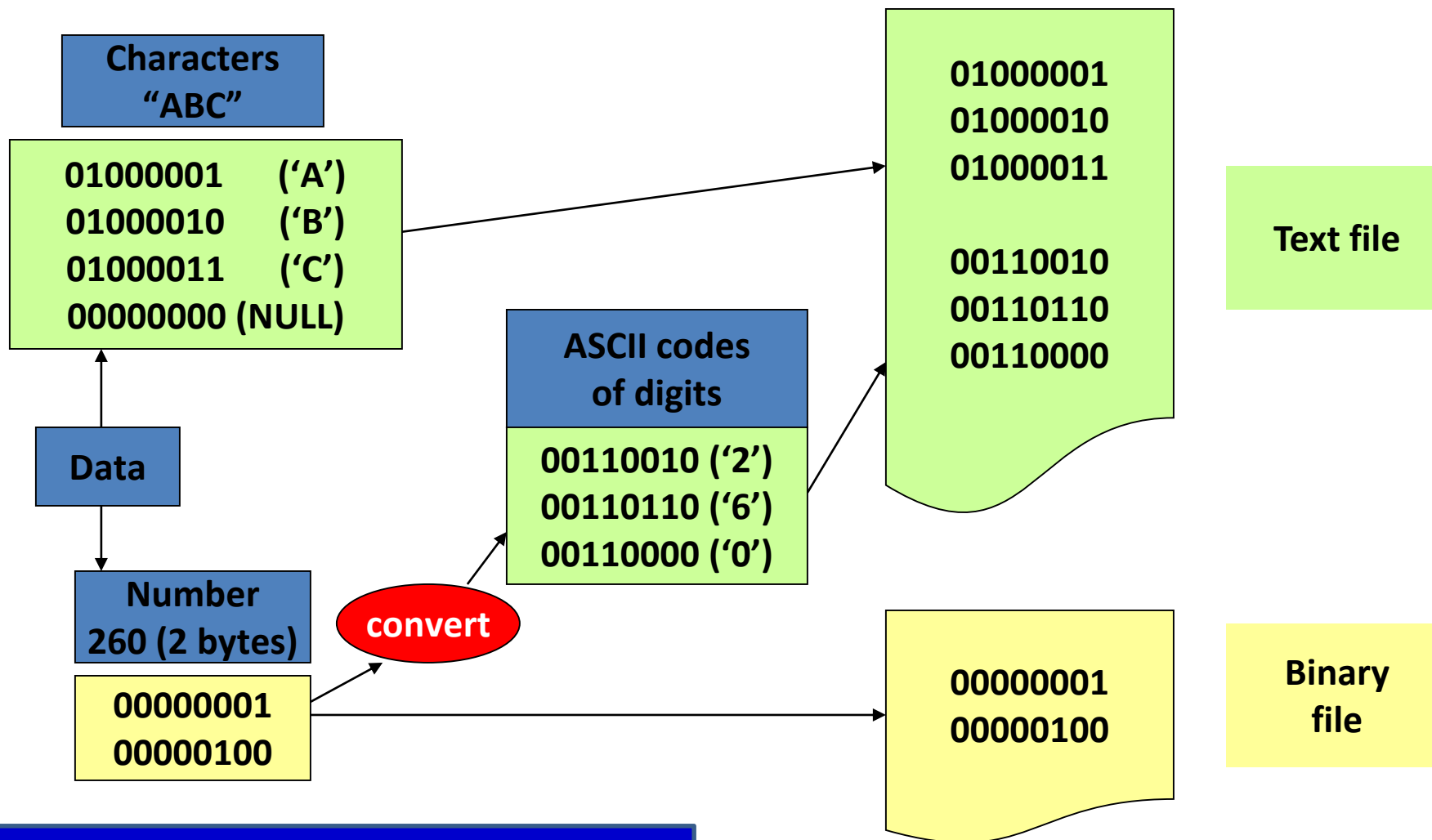


Binary Files

- ◆ A binary file contains data in **binary form (i.e. 0's and 1's)** instead of ASCII characters. They contain data that is stored in a similar manner to how it is stored in the main memory.
- The binary files can be created only from within a program and their contents can only be read by a program.
- More secure as they are not easily readable.
- They are generally stored with **.bin** file extension.



Type of Files in C (cont.)



Text format is more portable than binary format
But binary format is more efficient than text format

4. C File Operations

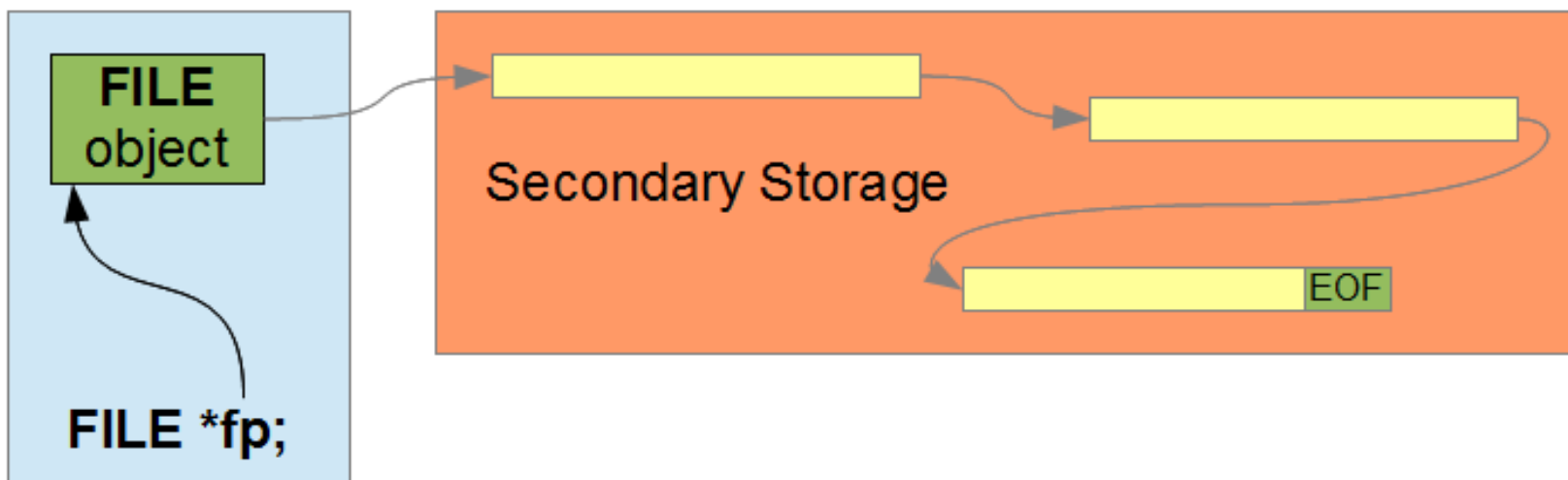
Operations on Files

There are different kinds of file operations in C:

1. Creating a new file
2. Opening an existing file
3. Reading from file
4. Writing to a file
5. Moving to a specific location in a file
6. Closing a file

4.1. Connection to File

- ◆ A C program connects to a file through an object of **FILE** type.
- ◆ We use a library function to retrieve the address of the file object, store that address in a pointer and subsequently access the file through that pointer called **File Pointer**.



4.1. Connection to File (cont.)

- ◆ Allocating a pointer to a FILE object takes the form:

```
FILE *identifier;
```

- ◆ Where FILE is the type of the FILE object and identifier is the name of the pointer to the FILE object. We call this pointer a handle to the object.
- ◆ The structure type FILE is declared in the **<stdio.h>** header file. To allocate memory for a FILE pointer, we write:

```
#include <stdio.h>

FILE *fp = NULL
```

4.2. Open a File in C

- ◆ For opening a file in C, the **fopen()** function is used with the filename or file path along with the required access modes.

- ◆ **Syntax:**

```
FILE* fopen(const char *file_name, const char *access_mode);
```

- ◆ **Parameters:**

- **file_name**: name of the file when present in the same directory as the source file. Otherwise, full path.
- **access_mode**: Specifies for what operation the file is being opened.

- ◆ **Return Value:**

- If the file is opened successfully, returns a file pointer to it.
- If the file is not opened, then returns NULL.

File opening modes in C

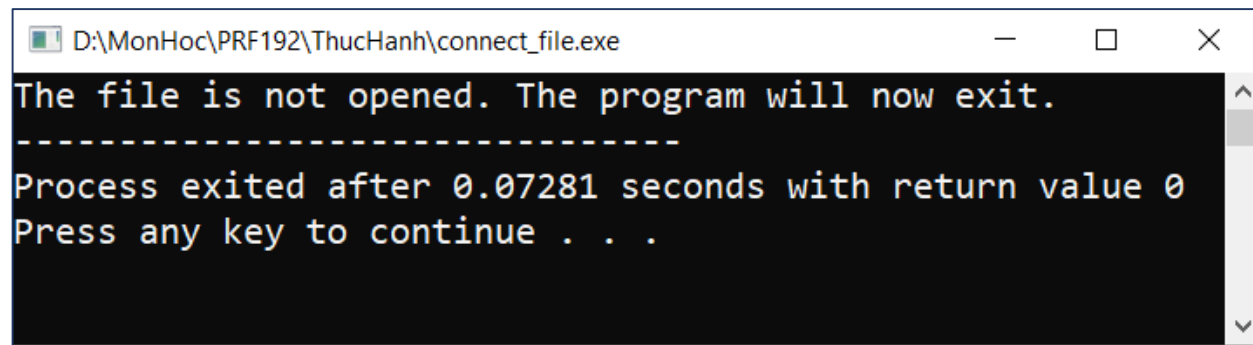
Mode	Description	Example
r	Opens a text file in read-only mode, allowing only reading operations.	<code>fopen("demo.txt", "r")</code>
w	When using the mode "w", <code>fopen()</code> initializes a text file for writing exclusively. If the file already exists, it clears its contents; otherwise, it creates a new file for writing.	<code>fopen("demo.txt", "w")</code>
a	When employing the "a" mode, <code>fopen()</code> enables opening a text file in append mode. This mode permits writing data to the end of the file, preserving existing content.	<code>fopen("demo.txt", "a")</code>
r+	When using the "r+" mode, <code>fopen()</code> facilitates opening a text file for both reading and writing operations. This mode grants the ability to manipulate data at any position within the file.	<code>fopen("demo.txt", "r+")</code>
w+	This mode opens a text file for both reading and writing. If the file with same name already exists, it truncates the file to zero length; otherwise, it creates a new file for both reading and writing operations.	<code>fopen("demo.txt", "w+")</code>

File opening modes in C (cont.)

Mode	Description	Example
a+	This mode opens a text file for both reading and writing, enabling data to be appended to the end of the file without overwriting existing content.	<code>fopen("demo.txt", "a+")</code>
rb	Opens a binary file in read-only mode, allowing reading operations on binary data.	<code>fopen("demo.bin", "rb")</code>
wb	Opens a binary file in write-only mode, truncating the file to zero length if it exists or creating a new file for writing binary data.	<code>fopen("demo.bin", "wb")</code>
ab+	This mode opens a binary file for both reading and writing operations, allowing binary data to be appended to the end of the file without overwriting existing content.	<code>fopen("demo.bin", "ab+")</code>
rb+	Opens a binary file for both reading and writing operations on binary data.	<code>fopen("demo.bin", "rb+")</code>
wb+	Opens a binary file for both reading and writing operations, truncating the file to zero length if it exists or creating a new file for reading and writing binary data.	<code>fopen("demo.bin", "wb+")</code>

Example of Opening a File

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  int main()
4  {
5      // file pointer variable to store the value returned by
6      // fopen
7      FILE* fptr;
8      // opening the file in read mode
9      fptr = fopen("filename.txt", "r");
10     // checking if the file is opened successfully
11     if (fptr == NULL) {
12         printf("The file is not opened. The program will "
13             "now exit.");
14         exit(0);
15     }
16     return 0;
17 }
```



```
D:\MonHoc\PRF192\ThucHanh\connect_file.exe
The file is not opened. The program will now exit.
-----
Process exited after 0.07281 seconds with return value 0
Press any key to continue . . .
```

4.3. Create a File in C

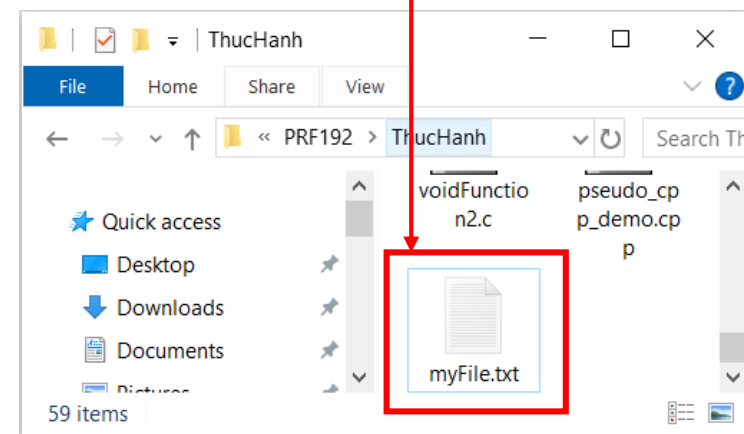
- ◆ The **fopen()** function can not only open a file but also can create a file if it does not exist already.
- ◆ Use the modes that allow the creation of a file if not found such as **w**, **w+**, **wb**, **wb+**, **a**, **a+**, **ab**, and **ab+**.
- ◆ Syntax:

```
FILE *fptr;  
fptr = fopen("filename.txt", "w");
```

Example of Create a File

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int main()
4 {
5     FILE* fptr;
6     // creating file using fopen() access mode "w"
7     fptr = fopen("myFile.txt", "w");
8     // checking if the file is created
9     if (fptr == NULL) {
10         printf("The file is not opened. The program will "
11              "exit now");
12         exit(0);
13     }
14     else {
15         printf("The file is created Successfully.");
16     }
17     return 0;
18 }
```

D:\MonHoc\PRF192\ThucHanh\cr...
The file is created Successfully.



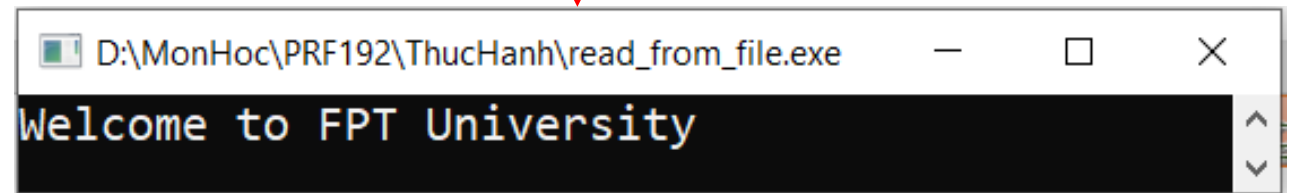
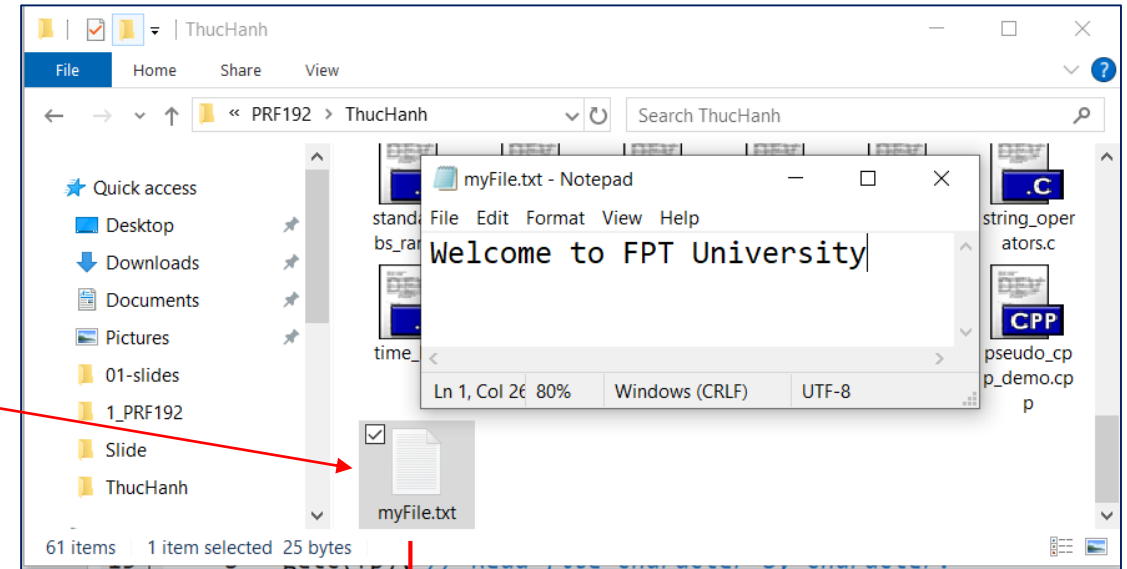
4.4. Reading From a File

- ◆ Using the functions to read data from a file

Mode	Description
<code>fscanf()</code>	Retrieves input from a file using a formatted string and variable argument list.
<code>fgets()</code>	Obtains a complete line of text from the file.
<code>fgetc()</code>	Reads a single character from the file.
<code>fgetw()</code>	Reads a numerical value from the file.
<code>fread()</code>	Extracts a specified number of bytes from a binary file.

4.4. Reading From a File: Example

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     FILE * fp;
6     char s;
7     fp = fopen("myFile.txt", "r");
8     if (fp == NULL) {
9         printf("\nCAN NOT OPEN FILE");
10        exit(1);
11    }
12    do {
13        s = getc(fp); // Read file character by character.
14        printf("%c", s);
15    }
16    while (s != EOF);
17    fclose(fp);
18
19    return 0;
20 }
```



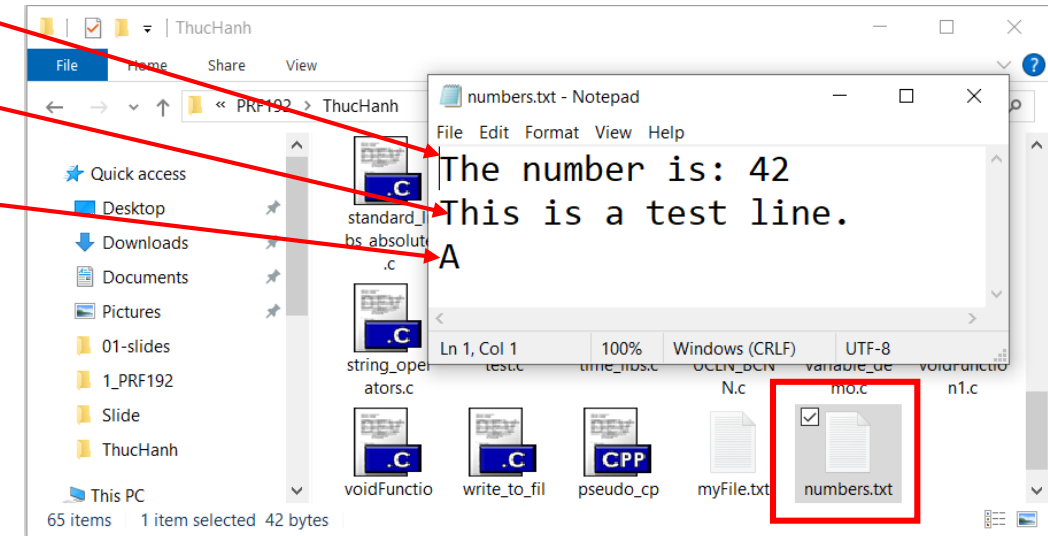
4.5. Write to a File

- ◆ Using the functions to **write** data to a file

Mode	Description
<code>fprintf()</code>	This function, akin to <code>printf()</code> , uses a formatted string and variable arguments list to print output to the file.
<code>fputs()</code>	Prints an entire line in the file along with a newline character at the end.
<code>fputc()</code>	Writes a single character into the file.
<code>putw()</code>	Writes a number to the file.
<code>fwrite()</code>	Writes the specified number of bytes to the binary file.

4.5. Write to a File: Example

```
3 int main() {  
4     FILE *fptr;  
5     int num = 42;  
6  
7     // Opening the file in write mode  
8     fptr = fopen("numbers.txt", "w");  
9  
10    // Writing data to the file using different functions  
11    fprintf(fptr, "The number is: %d\n", num);  
12    fputs("This is a test line.\n", fptr);  
13    fputc('A', fptr);  
14  
15    // Closing the file  
16    fclose(fptr);  
17  
18    return 0;  
19 }
```



Demo: Program Write and Read text file

The image shows a C program named `write_read_file.c` and its execution. The program's logic is as follows:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void inputClients(FILE *p, char *fileName);
5 void printClients(FILE *p, char *fileName);
6
7 int main() {
8     FILE *ptr;
9     // Write data to file
10    inputClients(ptr, "client.txt");
11    // Read data to file
12    printClients(ptr, "client.txt");
13    system("pause");
14    return 0;
15 }
```

The terminal window shows the input data being written to the file:

```
Enter the account, name, balance.
Enter EOF (Ctrl + Z) to end input.
# 6668 MinhTT 25.8889
# 8888 ThoPN3 13.057
# 1124 DungPT 999.123
# 2222 ThuyNT 12.45
# ^Z
Press any key to continue . . .
```

The File Explorer shows the directory structure, with `client.txt` highlighted in the `ThucHanh` folder.

The Notepad window shows the content of `client.txt` after reading:

```
6668 MinhTT 25.89
8888 ThoPN3 13.06
1124 DungPT 999.12
2222 ThuyNT 12.45
```

The terminal window shows the output of the `printClients` function:

Account	Name	Balance
6668	MinhTT	25.89
8888	ThoPN3	13.06
1124	DungPT	999.12
2222	ThuyNT	12.45

Press any key to continue . . .

Demo (cont.)

```
16 void inputClients(FILE *p, char *fileName){
17     // fopen opens file. Exit program if unable to create file
18     if((p = fopen(fileName, "w")) == NULL){
19         puts("File could not be opened");
20     }else{
21         puts("Enter the account, name, balance.");
22         puts("Enter EOF (Ctrl + Z) to end input.");
23         printf("%s", "# ");
24
25         unsigned int account;    // account number
26         char name[30];           // account name
27         double balance;          // account balance
28
29         // Input buffer
30         scanf("%d%29s%lf", &account, name, &balance);
31
32         // Write: account, name, balance into file with fprintf()
33         while(!feof(stdin)){
34             fprintf(p, "%d %s %.2lf\n", account, name, balance);
35             printf("%s", "# ");
36             scanf("%d%29s%lf", &account, name, &balance);
37         }
38         fclose(p); // fclose closes the file
39     }
40 }
```

Demo (cont.)

```
42 void printClients(FILE *p, char *fileName){
43     if((p = fopen(fileName, "r")) == NULL){
44         puts("File could not be opened");
45     }else{ // Read account, name, balance from file
46         unsigned int account; // account number
47         char name[30]; // account name
48         double balance; // account balance
49
50         printf("%-10s%-13s%s\n", "Account", "Name", "Balance");
51         fscanf(p, "%d%29s%lf", &account, name, &balance);
52
53         // While not end of file (EOF)
54         while(!feof(p)){
55             printf("%-10d%-13s%7.2lf\n", account, name, balance);
56             fscanf(p, "%d%29s%lf", &account, name, &balance);
57         }
58         fclose(p); // fclose closes the file
59     }
60 }
```

4.6. Write to a Binary File

- ◆ When writing data to a binary file, use the `fwrite()` function. This function allows us to store information in binary form, comprising sequences of bits (0s and 1s).

- ◆ **Syntax:**

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *file_pointer);
```

- ◆ **Parameters:**

- `ptr`: A reference to the memory block holding the data intended for writing.
- `size`: The byte size of each element to be written.
- `nmemb`: The count of elements to be written.
- `file_pointer`: The FILE pointer associated with the output file stream.

- ◆ **Return Value:** returns the number of objects successfully written

4.7. Reading from Binary File

- ◆ When retrieving data from a binary file, use the `fread()` function. This function facilitates the extraction of data from the file in its binary representation.

- ◆ **Syntax:**

```
size_t fread(void *memory_ptr, size_t size, size_t nmemb, FILE *file_pointer);
```

- ◆ **Parameters:**

- `memory_ptr`: A pointer to the memory block where the data will be stored.
- `size`: The size of each element to be read (in bytes).
- `nmemb`: The number of elements to be read.
- `file_pointer`: The FILE pointer pointing to the input file stream.

- ◆ **Return Value:** The function returns the number of objects successfully read.

Write and Read Binary File: Demo

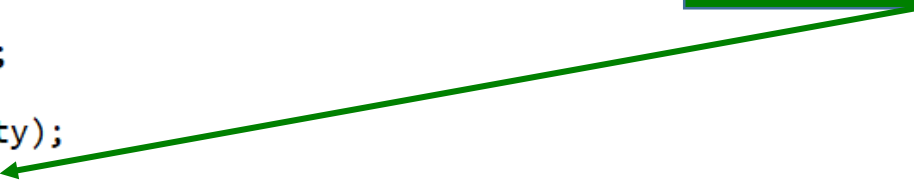
- ◆ Development of a program to manage 'n' product. Each product includes the following information: Product code, Price and quantity.
- ◆ **Requirements:**
 - Enter product information from the keyboard and save it to a binary file
 - Read data from the binary file and Print out a list of products

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Define the structure for product information
5  struct Product {
6      int product_id;
7      float price;
8      int quantity;
9  };
10
11 // Function Prototypes
12 void writeProducts(const char *filename, int numProducts);
13 void readProducts(const char *filename);
14
15 int main() {
16     const char *filename = "products.bin";
17     int numProducts;
18
19     printf("Enter the number of products: ");
20     scanf("%d", &numProducts);
21
22     // Write product data to the file
23     writeProducts(filename, numProducts);
24
25     // Read and display product data from the file
26     readProducts(filename);
27
28     return 0;
29 }
```

Write and Read Binary File: Demo

```
31 // Function to write products to a binary file
32 void writeProducts(const char *filename, int numProducts) {
33     FILE *filePtr;
34     struct Product product;
35
36     // Open the binary file for writing
37     if ((filePtr = fopen(filename, "wb")) == NULL) {
38         printf("Error! Failed to open the file for writing.\n");
39         exit(1);
40     }
41
42     // Input product details from the user and write to the file
43     for (int i = 0; i < numProducts; i++) {
44         printf("\nEnter details for product %d:\n", i + 1);
45         printf("Product ID: ");
46         scanf("%d", &product.product_id);
47         printf("Price: ");
48         scanf("%f", &product.price);
49         printf("Quantity: ");
50         scanf("%d", &product.quantity);
51
52         fwrite(&product, sizeof(struct Product), 1, filePtr);
53     }
54
55     printf("\nProducts have been written to the file successfully.\n");
56
57     // Close the file
58     fclose(filePtr);
59 }
```

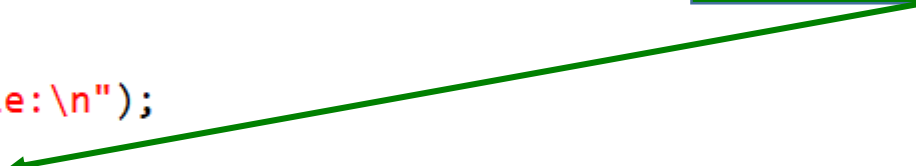
Write to binary file



Write and Read Binary File: Demo

```
61 // Function to read products from a binary file
62 void readProducts(const char *filename) {
63     FILE *filePtr;
64     struct Product product;
65
66     // Open the binary file for reading
67     if ((filePtr = fopen(filename, "rb")) == NULL) {
68         printf("Error! Failed to open the file for reading.\n");
69         exit(1);
70     }
71
72     printf("\nReading products from the file:\n");
73     // Read data from the binary file
74     while (fread(&product, sizeof(struct Product), 1, filePtr) == 1) {
75         // Print the product information
76         printf("Product ID: %d\tPrice: %.2f\tQuantity: %d\n",
77             product.product_id,
78             product.price,
79             product.quantity);
80     }
81
82     // Close the file
83     fclose(filePtr);
84 }
```

Read to binary file



Write and Read Binary File: Demo

```
D:\MonHoc\PRF192\ThucHanh\write_read_bin_file.exe

Enter the number of products: 3

Enter details for product 1:
Product ID: 101
Price: 10.99
Quantity: 50

Enter details for product 2:
Product ID: 102
Price: 15.49
Quantity: 30

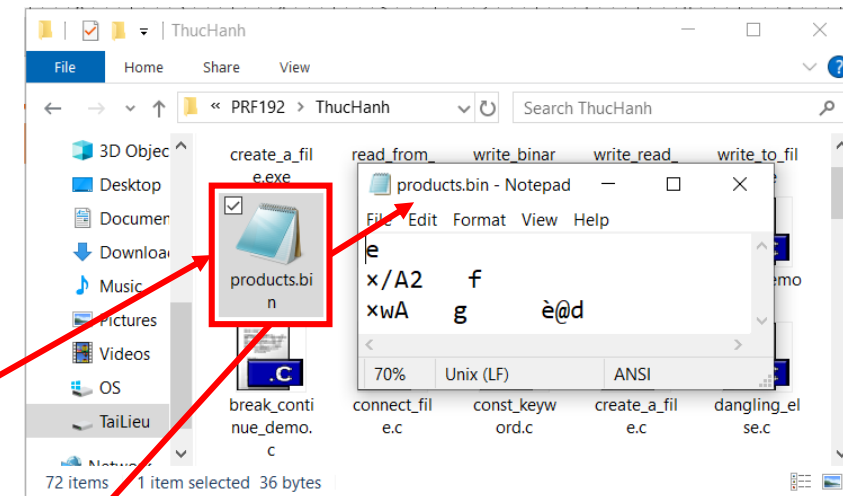
Enter details for product 3:
Product ID: 103
Price: 7.25
Quantity: 100

Products have been written to the file successfully.

Reading products from the file:
Product ID: 101 Price: 10.99    Quantity: 50
Product ID: 102 Price: 15.49    Quantity: 30
Product ID: 103 Price: 7.25     Quantity: 100
```

Input from
keyboard
and Write
to file

Read from file
and Print out



4.8. Moving to a specific location in a file: `fseek()`

- ◆ The **`fseek()`** function in C is used to move the file pointer to a specific location in a file.
- ◆ It allows you to read or write from any position in a file, making it a powerful tool for random-access file operations
- ◆ **Syntax:**

```
int fseek(FILE *file_ptr, long int offset, int pos);
```

- ◆ **Return Value:**
 - Returns 0 on success.
 - Returns a nonzero value on failure.

fseek()

◆ Parameters:

- **file_ptr**: A pointer to the file object (returned by **fopen()**).
- **offset**: The number of bytes to move the file pointer.
- **pos**: Determines the position from where the offset is applied. It can be:
 - **SEEK_SET**: Beginning of the file.
 - **SEEK_CUR**: Current position of the file pointer.
 - **SEEK_END**: End of the file.

◆ Common Use Cases:

- Jump to a specific location in a file.
- Skip over sections of a file.
- Read or write from a specific position in a file.

rewind() function

- ◆ The **rewind()** function in C is used to move the file pointer back to the beginning of a file.
- ◆ Unlike **fseek()**, which requires parameters, **rewind()** is a simple way to reset the file pointer to the start of a file.
- ◆ **Syntax:**

```
void rewind(FILE *file_pointer);
```

- ◆ **Parameter:**
 - **file_pointer**: A pointer to the file object, which must have been opened by **fopen()**
- ◆ **Return Value:** does not return a value

Demo: `rewind(FILE*)`

```
1 /*test_rewind.c */
2 #include <stdio.h>
3 int main()
4 {   char fname[] = "test_rewind.txt";
5     char c; /* a character from file */
6     int i;
7     FILE * f= fopen(fname, "r");
8     printf("10 first characters:\n");
9     for (i=0;i<10;i++) putchar(fgetc(f));
10    rewind(f);
11    printf("\n\nAfter rewind:\n");
12    while ((c=fgetc(f))!=EOF) putchar(c);
13    fclose(f);
14    getchar();
15    return 0;
16 }
```

test_rewind.txt
content for testing rewind function

test_rewind.txt
content for testing rewind function

C:\K:\GiangDay\FU\OOP\BaiTap\test_rewind...
10 first characters:
content fo

After rewind:
content for testing rewind function

Demo: fseek(...)

test_fseek.txt

content for testing fseek function

```
/*test_fseek.c */
#include <stdio.h>
int main()
{
    char fname[] = "test_fseek.txt";
    char c; /* a character from file */
    int i;
    FILE * f= fopen(fname, "r");
    printf("15 first characters:\n");
    for (i=0;i<15;i++) putchar(fgetc(f));
    puts("\n");
    fseek(f,-5,SEEK_CUR); /* from CURRENT position */
    for (i=0;i<5;i++) putchar(fgetc(f));
    puts("\n");
    fseek(f,-10,SEEK_END); /* from END position */
    for (i=0;i<5;i++) putchar(fgetc(f));
    puts("\n");
    fseek(f,10,SEEK_SET); /* from BEGINNING position */
    for (i=0;i<5;i++) putchar(fgetc(f));
    fclose(f);
    getchar();
    return 0;
}
```

content for testing fseek function

content for testing fseek function

content for testing fseek function

content for testing fseek function

content for testing fseek function

EOF
(2bytes)

content for testing fseek function

content for testing fseek function

```
15 first characters:
content for tes

r tes

funct

r tes_
```

Summary

- ◆ Understand what a file is? The role of files in storing data on secondary memory.
- ◆ Types of Files in C
- ◆ Using C language to work with Files
- ◆ Open/Close files
- ◆ Read and write data to/from text files and binary files
- ◆ Move the pointer to work with data in files